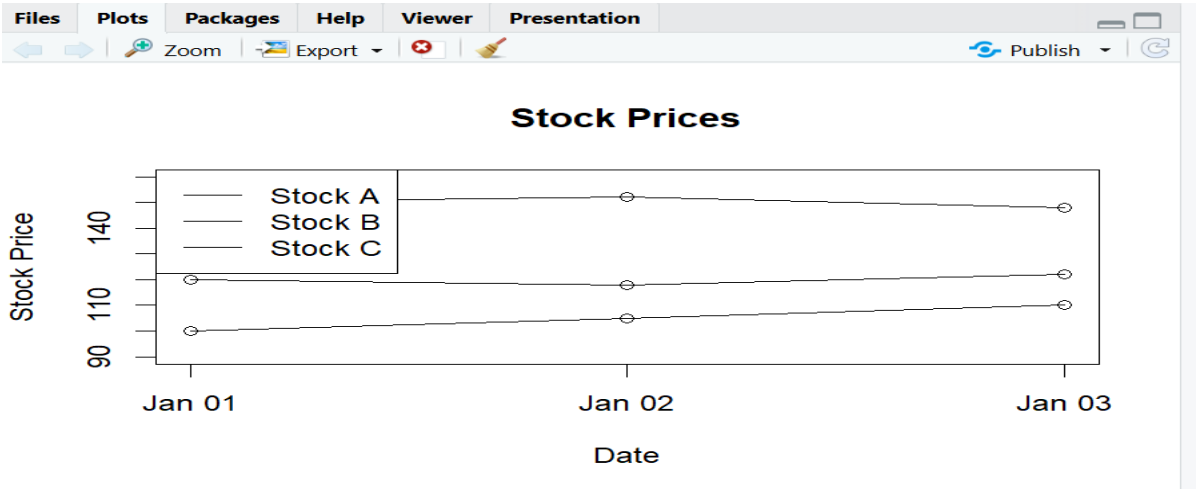
SET 16:

1.Create a line chart to visualize the stock prices of three companies (Stock A, Stock B, and Stock C) over a specific time period. Label the axes and title the chart.

Code:

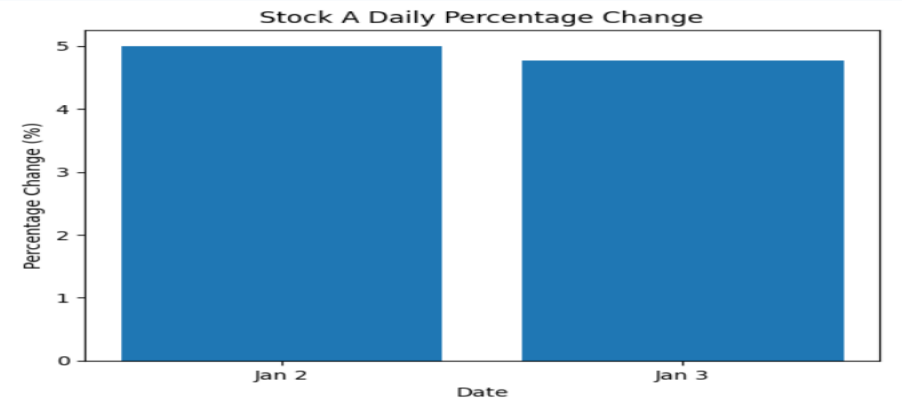
```
data <- data.frame(
 Date=as.Date(c("2023-01-01","2023-01-02","2023-01-03")),
 A=c(100,105,110), B=c(150,152,148), C=c(120,118,122))
plot(data$Date,data$A,type="o",ylim=c(90,160),
     xlab="Date",ylab="Stock Price",main="Stock Prices")
lines(data$Date,data$B,type="o")
lines(data$Date,data$C,type="o")
legend("topleft",c("Stock A","Stock B","Stock C"),lty=1)
```



2. Generate a bar chart showing the daily percentage change in stock prices for Stock A. Label the chart elements.

Code:

```
import matplotlib.pyplot as plt
a = [100,105,110]
change = [(a[i]-a[i-1])/a[i-1]*100 for i in range(1,len(a))]
plt.bar(["Jan 2","Jan 3"],change)
plt.xlabel("Date")
plt.ylabel("Percentage Change (%)")
plt.title("Stock A Daily Percentage Change")
plt.show()
```



3. Build a table to display the stock price data for each company over the given period. Label the table elements.

Code:

```
import pandas as pd

df = pd.DataFrame({

"Date":["2023-01-01","2023-01-02","2023-01-03"],

"Stock A":[100,105,110],

"Stock B":[150,152,148],

"Stock C":[120,118,122]})

print(df)
```

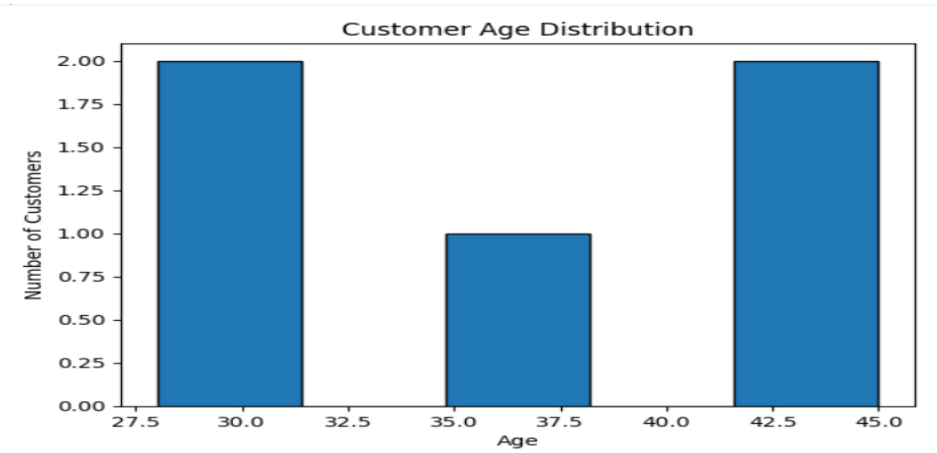
```
         Date  Stock A  Stock B  Stock C
0  2023-01-01      100      150      120
1  2023-01-02      105      152      118
2  2023-01-03      110      148      122
```

SET 17

1.In Python, create a histogram to visualize the distribution of customer ages. Label the axes and title the chart.

Code:

```
import matplotlib.pyplot as plt
age = [28,35,42,30,45]
plt.hist(age,bins=5,edgecolor="black")
plt.xlabel("Age")
plt.ylabel("Number of Customers")
plt.title("Customer Age Distribution")
plt.show()
```

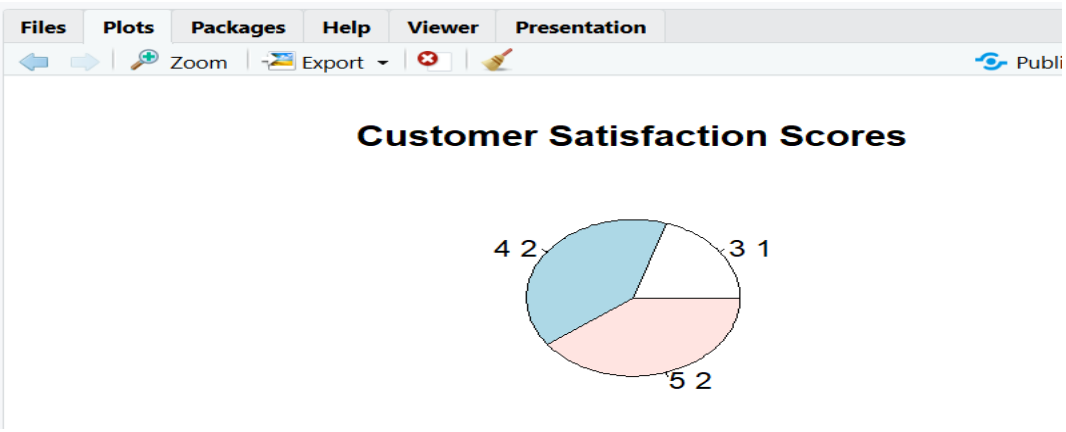


2. In R, generate a pie chart to represent the distribution of overall customer satisfaction scores.

Code:

Include labels.

```
score <- c(4,5,3,4,5)
count <- table(score)
pie(count,
    labels=paste(names(count),count),
    main="Customer Satisfaction Scores")
```



4. In Python, perform sentiment analysis on open-ended customer feedback and create a word cloud visualization.

Code:

```
from textblob import TextBlob

from wordcloud import WordCloud

import matplotlib.pyplot as plt

feedback = "Excellent service and friendly staff. Very happy with the product. Good experience."

sentiment = TextBlob(feedback).sentiment.polarity

print("Sentiment:", sentiment)

wordcloud = WordCloud(width=800,height=400).generate(feedback)

plt.imshow(wordcloud)

plt.axis("off")

plt.title("Customer Feedback Word Cloud")

plt.show()
```

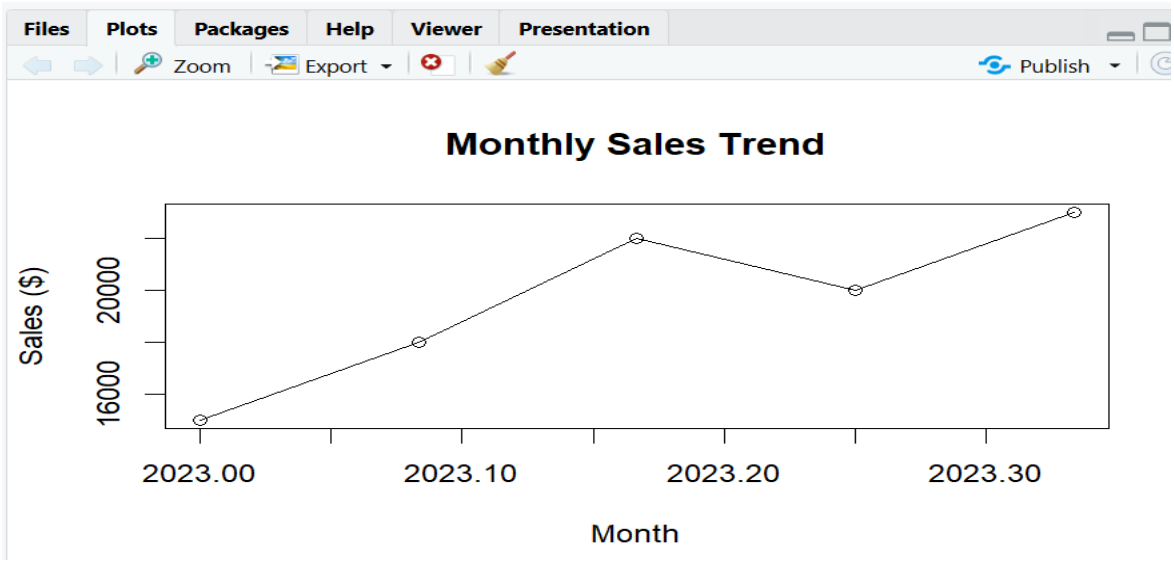
SET 18

5. In R, create a time series line chart to visualize the trend in monthly sales. Label the axes and

title the chart.

Code:

```
sales <- c(15000,18000,22000,20000,23000)

ts_data <- ts(sales,start=c(2023,1),frequency=12)

plot(ts_data,type="o",

    xlab="Month",ylab="Sales ($)",

    main="Monthly Sales Trend")
```



6. In Python, generate a scatter plot to analyze the relationship between advertising budget and

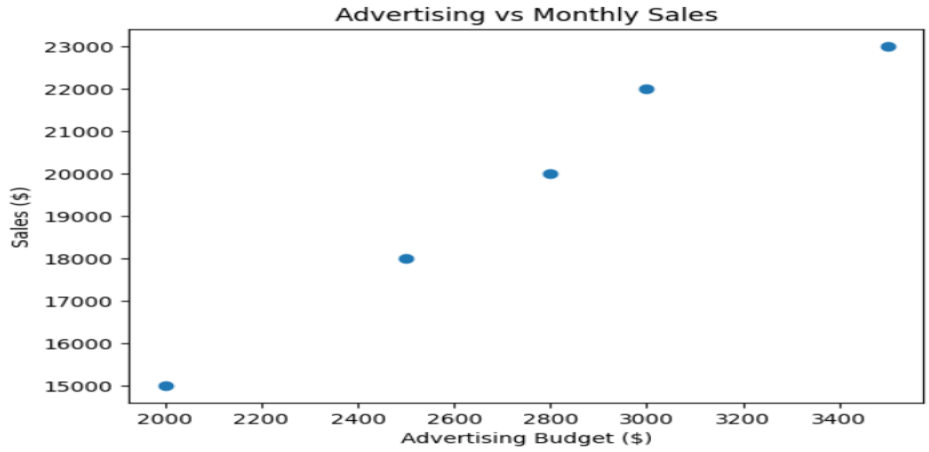
monthly sales. Explain any insights.

Code:

```
import matplotlib.pyplot as plt

advertising = [2000,2500,3000,2800,3500]

sales = [15000,18000,22000,20000,23000]

plt.scatter(advertising,sales)

plt.xlabel("Advertising Budget ($)")

plt.ylabel("Sales ($)")
```

plt.title("Advertising vs Monthly Sales")

plt.show()

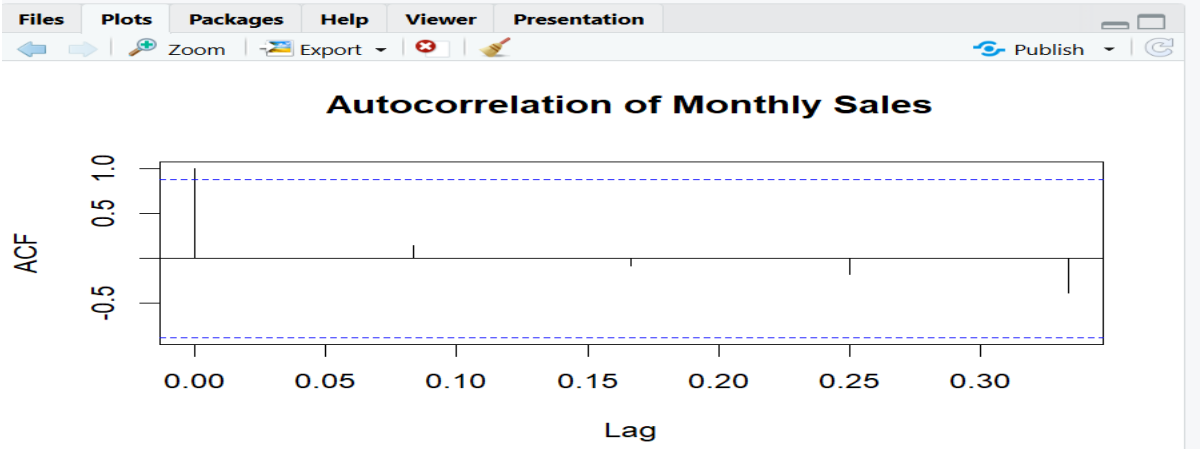


8. In R, create an autocorrelation plot to identify seasonality in the time series data

Code:

sales <- ts(c(15000,18000,22000,20000,23000),frequency=12)

acf(sales,main="Autocorrelation of Monthly Sales").

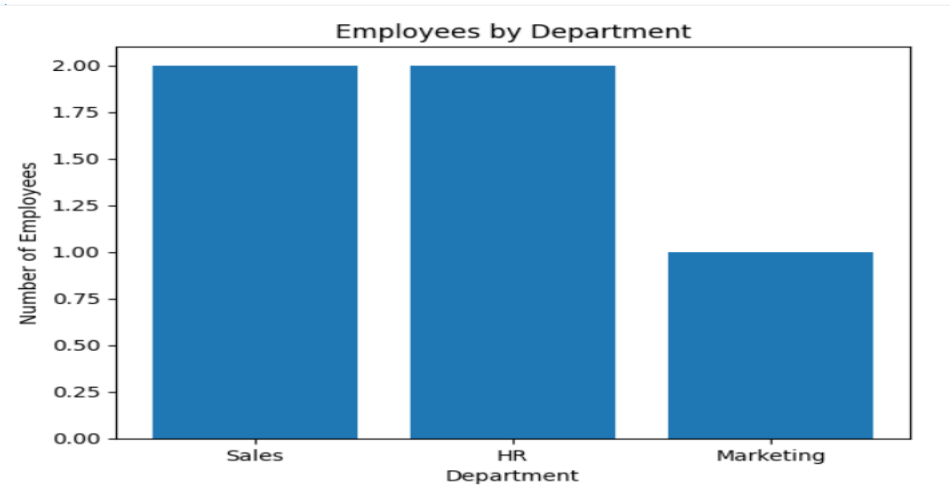


SET 19

9.In Python, create a bar chart to visualize the distribution of employees across different departments. Label the chart elements.

Code:

```
import matplotlib.pyplot as plt
from collections import Counter
dept = ["Sales","HR","Marketing","Sales","HR"]
count = Counter(dept)

plt.bar(count.keys(),count.values())
plt.xlabel("Department")
plt.ylabel("Number of Employees")
plt.title("Employees by Department")
plt.show()
```



10. In R, generate a line chart to visualize the performance trend of employees over time. Label

the axes and title the chart.

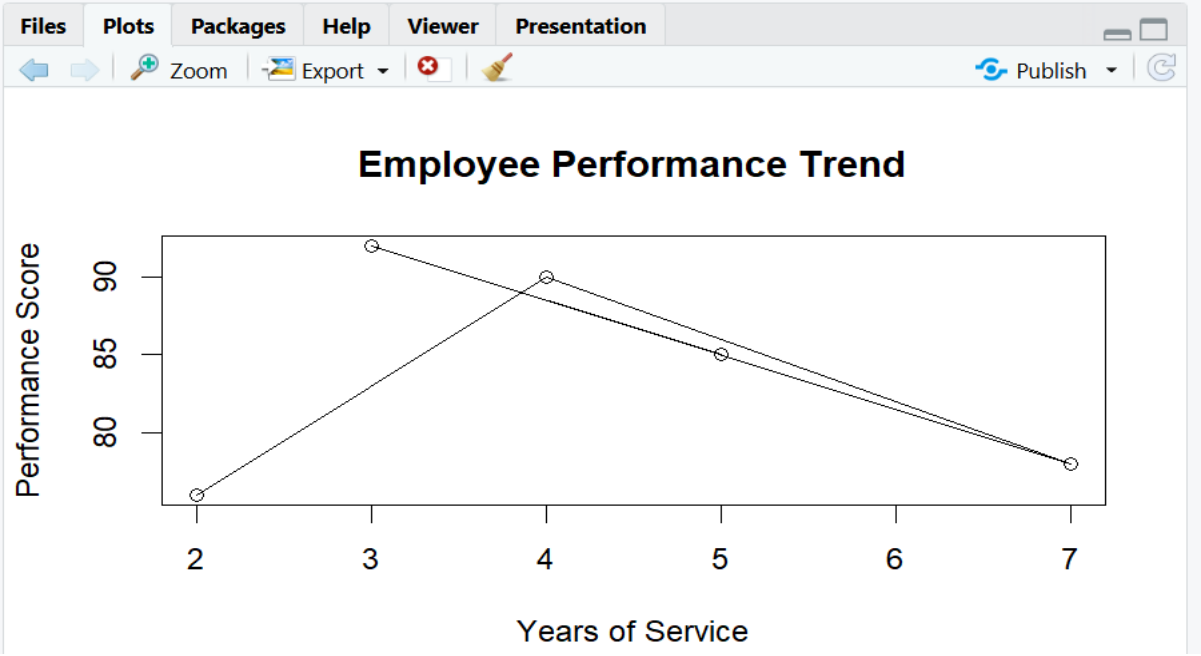
Code:

```
years <- c(5,3,7,4,2)
score <- c(85,92,78,90,76)
plot(years,score,type="o",
```

xlab="Years of Service",

ylab="Performance Score",

main="Employee Performance Trend")



12. In Python, develop a table showing the performance data for each employee. Label the table

Code:

elements.

import pandas as pd

df = pd.DataFrame({

"Employee ID":[1,2,3,4,5],

"Department":["Sales","HR","Marketing","Sales","HR"],

"Years":[5,3,7,4,2],

"Performance":[85,92,78,90,76]})

print(df)

```
   Employee ID Department  Years  Performance
0            1      Sales      5           85
1            2         HR      3           92
2            3  Marketing      7           78
3            4      Sales      4           90
4            5         HR      2           76
```

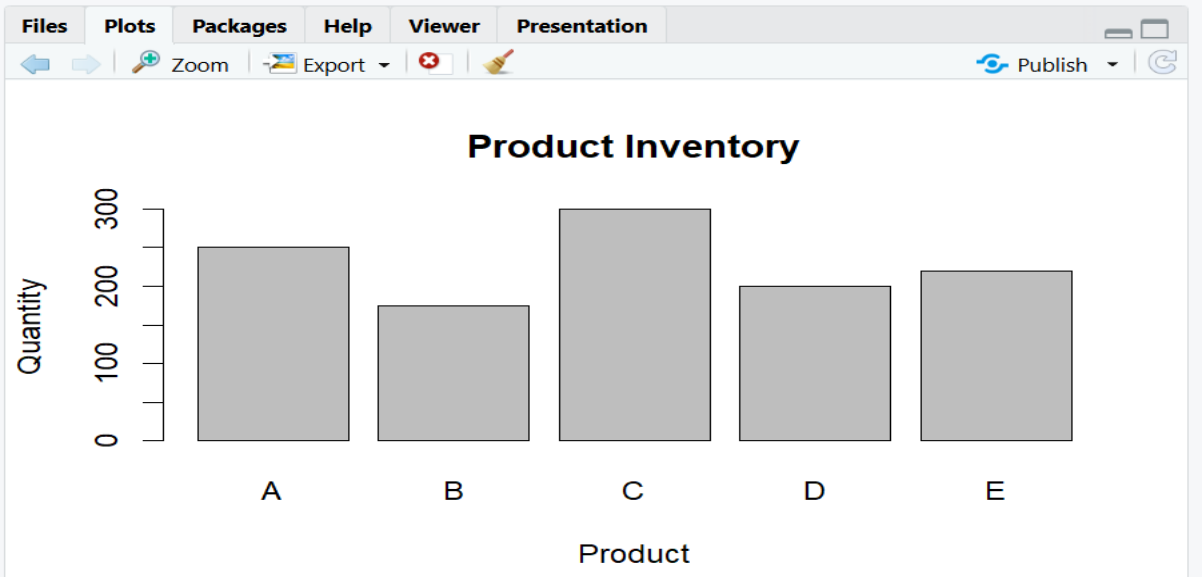
SET 20

13. In R, create a bar chart to visualize the quantity of each product in the inventory. Label the

axes and title the chart.

Code:

```
product <- c("A","B","C","D","E")

quantity <- c(250,175,300,200,220)

barplot(quantity,names.arg=product,

    xlab="Product",ylab="Quantity",

    main="Product Inventory")
```



14. In Python, generate a stacked bar chart to show the quantity of each product within different

product categories.

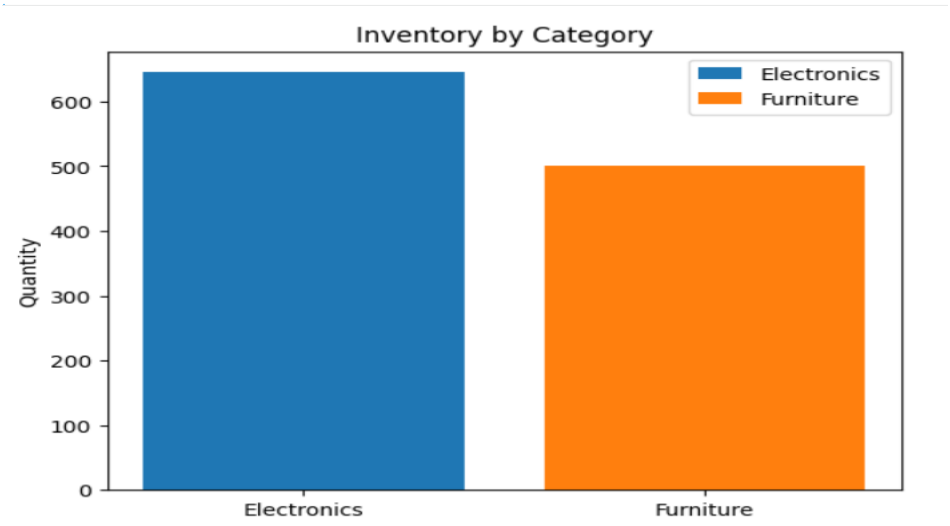
Code:

```
import matplotlib.pyplot as plt

electronics = [250,175,220]

furniture = [300,200]

plt.bar(["Electronics"],[sum(electronics)],label="Electronics")

plt.bar(["Furniture"],[sum(furniture)],label="Furniture")

plt.ylabel("Quantity")
```

plt.title("Inventory by Category")

plt.legend()

plt.show()



16. In Python, create a scatter plot to explore the relationship between product price and quantity

Code:

available. Explain any insights.

import matplotlib.pyplot as plt

price = [500,300,700,450,600]

quantity = [250,175,300,200,220]

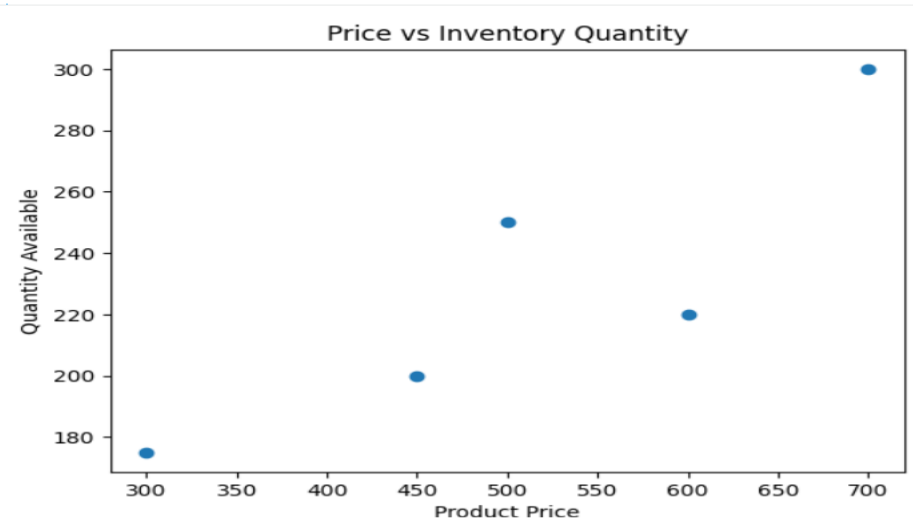
plt.scatter(price,quantity)

plt.xlabel("Product Price")

plt.ylabel("Quantity Available")

plt.title("Price vs Inventory Quantity")

plt.show()


Price vs Inventory Quantity
Quantity Available
300
280
260
240
220
200
180
300
350
400
450
500
550
600
650
700
Product Price

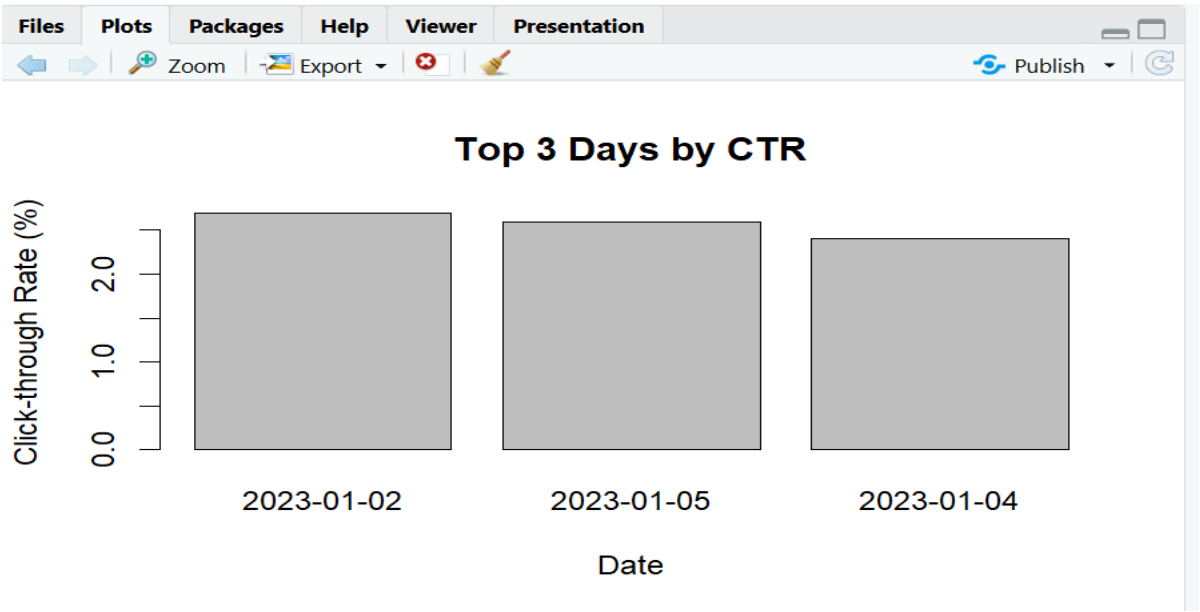
SET 21

18. In R, generate a bar chart to show the top N days with the highest click-through rates. Label

the chart elements.

Code:

```
data <- data.frame(
 Date=as.Date(c("2023-01-01","2023-01-02","2023-01-03",
              "2023-01-04","2023-01-05")),
 CTR=c(2.3,2.7,2.0,2.4,2.6))
top <- head(data[order(-data$CTR),],3)
barplot(top$CTR,names.arg=top$Date,
      xlab="Date",ylab="Click-through Rate (%)",
      main="Top 3 Days by CTR")
```



19. In Python, build a stacked area chart to display the distribution of user interactions (likes, shares, comments) on the website.

Code:

```
import matplotlib.pyplot as plt


dates = ["Jan 1","Jan 2","Jan 3","Jan 4","Jan 5"]
```

```
likes = [500,600,450,650,700]
shares = [150,180,130,170,200]
comments = [80,100,70,90,120]
plt.stackplot(dates,likes,shares,comments,
              labels=["Likes","Shares","Comments"])
plt.xlabel("Date")
plt.ylabel("Interactions")
plt.title("Website User Interactions")
plt.legend()
plt.show()
```